一、ORM(2)

二、教学目标

1、掌握单表查询

2、掌握双下划线的操作

3、掌握一对一/一对多/多对多的创建

4、写入初始数据和使用案例插入数据

三、单表查询

- values
  - 返回Queryset对象,可以看作是列表套字典的形式

```
#values 相当于select后面的字段
res = models.User.objects.values('name','age')
print(res) # 返回Queryset对象,可以看作是列表套字典的形式
```

- values_list
  - 返回Queryset对象,可以看作是列表套元组的形式

```
# values_list 相当于select后面的字段 没有键 只有数据
res1 = models.User.objects.values_list('name', 'age')
print(res1)  # 返回Queryset对象,可以看作是列表套元组的形式
```

- distinct
  - 去重

```
# distinct 去重
res = models.User.objects.values('name','age').distinct()
print(res)
```

- order_by和reverse
  - 升序和降序

```
#order_by 排序
res = models.User.objects.values('age').order_by('age') #默认为升序 从小到大
print(res)
#降序 第一种方式: -   从大到小
res = models.User.objects.values('age').order_by('-age')
print(res)
#降序 第二种方式: reverse()
res = models.User.objects.values('age').order_by('age').reverse() #反转
print(res)
```

- count
  - 统计总数

```
#count 返回数据条数
res = models.User.objects.count()
print(res)
```

- exclude
  - 条件取反

```
# exclude(条件) 对条件取反 select * from 表 where not 条件;
res = models.User.objects.exclude(name='admin')
print(res)
```

- exists
  - 条件数据是否存在,返回布尔值

```
#exists() 条件数据是否存在,返回布尔值
res = models.User.objects.filter(name='admin123').exists()
print(res)
```

- **总结**
  - all()： 获取所有数据,返回 Queryset 对象
  - filter()： 获取满足条件的数据,返回Queryset对象
  - get()： 直接获取满足条件的对象,如果条件不成立 立刻报错
  - first()： 获取QuerySet对象中的第一个元素
  - last()： 获取QuerySet对象中的最后一个元素
  - values()： 获取指定字段的数据,返回 QuerySet对象,格式类似于列表套字典
  - values_list()： 获取指定字段的数据,返回 QuerySet对象,格式类似于列表套元组
  - distinct()：去重
  - order_by()：排序
  - reverse：反转
  - count()：返回数据条数
  - exclude()：对条件取反
  - exists() : 条件数据是否存在,返回布尔值

四、双下划线

- __gt
  - 大于
- __lt
  - 小于
- __gte
  - 大于等于
- ___lte
  - 小于等于

```
#查询年龄大于20岁的信息
res = models.User.objects.filter(age__gt=20) #age > 20
print(res)

# 查询年龄小于20岁的信息
res = models.User.objects.filter(age__lt=20)  # age < 20
print(res)

# 查询年龄大于等于20岁的信息
res = models.User.objects.filter(age__lt=20)  # age >= 20
print(res)

# 查询年龄小于等于20岁的信息
res = models.User.objects.filter(age__lt=20)  # age <= 20
print(res)
```

- __in
  - 用于筛选字段值在某个列表中的记录

```
#查询年龄为20或者21的信息
res = models.User.objects.filter(age__in=(19,21))
print(res)
```

- __range
  - 类似数据库里的between and

```
# 查询年龄为20到21之间的信息
res = models.User.objects.filter(age__range=(19, 21))
print(res)
```

- __contains
  - 模糊查询 默认区分大小写
- __icontains
  - 模糊查询 默认不区分大小写

```
#__contains 区分大小写
#__icontains 不区分大小写
res = models.User.objects.filter(name__contains='a')
print(res)
res = models.User.objects.filter(name__icontains='a')
print(res)
```

- __startswith
  - 匹配字段值以指定字符串开头的记录 类似于 LIKE 'value%'
- __endswith
  - 匹配字段值以指定字符串结尾的记录 类似于 LIKE '%value'

```
# __startswich 匹配字段值以指定字符串开头的记录 类似于 LIKE 'value%'
# __endswith 匹配字段值以指定字符串结尾的记录 类似于 LIKE '%value'
res = models.User.objects.filter(name__startswith='a')
print(res)
res = models.User.objects.filter(name__endswith='i')
print(res)
```

- __year
  - 提取日期/时间字段的 年份，并匹配指定值
- __month
  - 提取日期/时间字段的 月份（1-12），并匹配指定值。
- __day
  - 提取日期/时间字段的 日（1-31），并匹配指定值。

```
#__year 年
res = models.User.objects.filter(create_time__year=2025)
print(res)
# __year 月
res = models.User.objects.filter(create_time__month=5)
print(res)
# __year 日
res = models.User.objects.filter(create_time__day=15)
print(res)
```

- 总结
  - 数据库表:django默认以小写的应用名_小写模型类名作为数据库表名
- 创建模型类时,如果没有指定属性作为主键字段,django会自动创建自增长的主键名称,名称为id
- 如果手动创建了主键字段,就不会帮你创建了,可以使用 primary_key 代替主键字段名
- 字段类型
  - AutoField:自增长整型,一般用于主键字段
  - BooleanField:布尔值,值为 True 或者 False
  - NullBooleanField:布尔值,值为True 或者 False 或者Null

    - CharField:字符串类型,需要指定最大长度,max_length
    - TextField:文本类型,一般超过 4000 字符使用
    - IntegerField:整型,最大长度是 10 位
    - BigIntegerField:大整型,最大长度是 38 位
    - FloatField:浮点型
    - DecimalField:十进制类型(需要指定max_disgits(总位数)和 decimal_places(小数位数))
    - DateField:日期(年月日)
    - TimeField:时间(时分秒)
    - DateTimeField:日期时间(年月日时分秒)
        - auto_now:每次保存自动更新为当前时间
        - auto_now_add:创建时保存当前时间,后面除非人为修改,不在变化
    - EmailField:邮箱格式数据,对于数据库来说就是 varchar(254)
    - FileField:文件数据
    - ImageField:继承 FileField,保存图片数据
- 字段参数
    - null:是否允许为空,默认为 True
    - unique:是否唯一,默认为 False
    - primary_key:主键
    - default:默认值
    - verbose_name:注释.不写默认以属性名
    - db_column:数据库字段名,不写默认以属性名作为字段名
    - db_index:设置索引,默认为 False

五、关系字段

Foreignkey:外键字段,用来关联其他的模型类

- to:要关联模型类（表）
- to_field:要关联的字段(非必要),默认关联关联表的主键字段
- on_delete:删除关联表数据时,当前表的操作,一般使用 models.CASCADE (级联删除)

OneToOneField:

- 1对1关系字段,参数同Foreignkey一模一样

ManyToManyField:

- 多对多关系字段,默认情况下会自动创建第三张关系表
    - to: 要关联的模型类

5.1 1 一对一/一对多/多对多

- app01应用中models.py 补充代码

```
#一对多 -> 一个出版社可以出多种图书 一个书只能有一个出版社出版
#多对多 -> 一本书有多个作者 一个作者是不是也可以写多本书
#一对一 -> 一个作者一个详情
```

```
#图书表
class Book(models.Model):
    title = models.CharField(max_length=64) #书名
    price = models.DecimalField(max_digits=7,decimal_places=2) #价格
    publish_date = models.DateField(auto_now_add=False) #出版日期
    #出版社和图书是一对多的关系
    # 创建之后会自动关联publish的主键字段,
    # 同时创建的字段名后面会加上_id,变为publish_id
    publish = models.ForeignKey(to='Publish',on_delete=models.CASCADE)
    #图书和作者是多对多的关系
    authors = models.ManyToManyField(to='Author')
    #authors 这个属性就是指向第三张关系的地址
#出版社表
class Publish(models.Model):
    name = models.CharField(max_length=32) #出版社名称
    address = models.CharField(max_length=54) #出版社地址
    email = models.EmailField()  #出版社邮箱
#作者表
class Author(models.Model):
    name = models.CharField(max_length=32) #作者名字
    age = models.IntegerField() #作者年龄


#作者详情表
class AuthorDetail(models.Model):
    phone = models.BigIntegerField() #手机号
    addr = models.CharField(max_length=64) #地址

    author = models.OneToOneField(to='Author',on_delete=models.CASCADE)
```

- 生成迁移文件

```
python manage.py makemigrations
```

- 执行迁移

```
python manage.py migrate
```

5.2 2 初始数据

`app01_publish`

```
INSERT INTO `django_orm`.`app01_publish`(`id`, `name`, `address`, `email`)
VALUES (1, '东方出版社', '东方', '123@163.com');
INSERT INTO `django_orm`.`app01_publish`(`id`, `name`, `address`, `email`)
VALUES (2, '南方出版社', '南方', '456@163.com');
```

`app01_book`

```sql
INSERT INTO `django_orm`.`app01_book`(`id`, `title`, `price`, `publish_date`, `publish_id`) VALUES (1, '三国演义', 36.36, '1999-01-02', 1);
INSERT INTO `django_orm`.`app01_book`(`id`, `title`, `price`, `publish_date`, `publish_id`) VALUES (2, '水浒传', 37.37, '2000-01-01', 1);
INSERT INTO `django_orm`.`app01_book`(`id`, `title`, `price`, `publish_date`, `publish_id`) VALUES (3, '西游记', 40.11, '2004-01-02', 2);
INSERT INTO `django_orm`.`app01_book`(`id`, `title`, `price`, `publish_date`, `publish_id`) VALUES (4, '红楼梦', 66.72, '2008-08-08', 2);
```

`app01_authordetail`

```sql
INSERT INTO `django_orm`.`app01_authordetail`(`id`, `phone`, `addr`, `author_id`) VALUES (1, 13008833777, '上海 ', 1);
INSERT INTO `django_orm`.`app01_authordetail`(`id`, `phone`, `addr`, `author_id`) VALUES (2, 13677778888, '北京', 2);
INSERT INTO `django_orm`.`app01_authordetail`(`id`, `phone`, `addr`, `author_id`) VALUES (3, 15588889999, '深圳', 3);
```

`app01_author`

```sql
INSERT INTO `django_orm`.`app01_author`(`id`, `name`, `age`) VALUES (1, 'zhangsan', 20);
INSERT INTO `django_orm`.`app01_author`(`id`, `name`, `age`) VALUES (2, 'lisi', 26);
INSERT INTO `django_orm`.`app01_author`(`id`, `name`, `age`) VALUES (3, 'wangwu', 20);
```

`app01_book_authors`

```sql
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (1, 1, 1);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (2, 1, 2);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (3, 2, 2);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (4, 2, 3);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (5, 3, 1);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (6, 3, 2);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (7, 4, 1);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (8, 4, 2);
INSERT INTO `django_orm`.`app01_book_authors`(`id`, `book_id`, `author_id`) VALUES (9, 4, 3);
```

5.3 3 插入数据

```
#一对多的增删改查
#增
#第一种方式: 通过往外键字段中插入值
res = models.Book.objects.create(title='聊斋志异',
price=30.99,
publish_date='2008-01-01',
publish_id=1
)
print(res)
#第二种方式: 虚拟对象
publish_obj =models.Publish.objects.filter(pk=2).first()
res1 = models.Book.objects.create(title='聊斋志异2',
price=30.99,
publish_date='2008-02-08',
publish=publish_obj
)
print(res1)
```

六、作业

6.1 完成本天的案例